#06/01/2025

Angular Authentication With JSON Web Tokens (JWT)

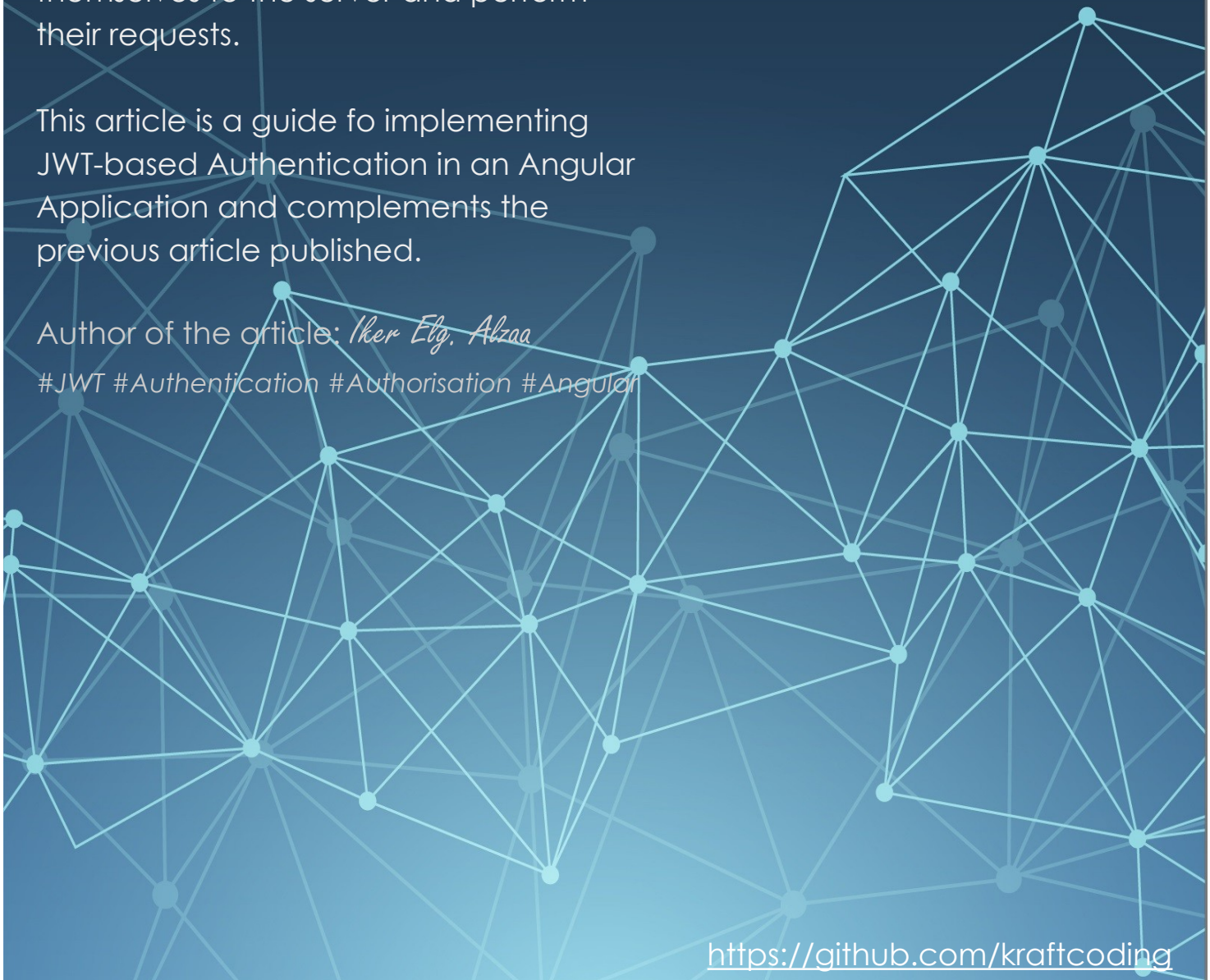
Source code here
Back-end related article here

Transmitting data across multiple systems can be risky, as this data can be intercepted and modified before it gets to the receiver. A JSON Web Token is a tool that allows users to identify themselves to the server and perform their requests.

This article is a guide fo implementing JWT-based Authentication in an Angular Application and complements the previous article published.

Author of the article: *Iker Elg. Alzaa*

#JWT #Authentication #Authorisation #Angular



https://github.com/kraftcoding

Transmitting data across multiple systems can be risky, as this data can be intercepted and modified before it gets to the receiver. The sender must therefore take steps to ensure that the data is securely transmitted, and the receiver must be able to confirm that the data is from the expected sender and that it was not modified during transport.

JWT helps solve these challenges by providing an elegant, secure way of transferring statements (referred to as "claims") about an entity (typically the user) between two parties.

One big advantage is that they're self-contained. This means all the user data the website needs is packed right into the token. Plus, JWTs are super compact, which helps keep things fast and efficient.

Building Blocks

JWTs are made up of three parts: *a header, a payload, and a signature*. Here's what each part does:

- *Header:* This part tells us what kind of token we're dealing with (which is JWT) and the algorithm used for creating the signature.
- *Payload:* This is where the user data is stored. It's also where you'll find details about when the token was issued and when it will expire.
- *Signature:* Think of the signature as a safety seal. It's created by taking the encoded header, the encoded payload, and a secret key, then running them through a special algorithm. The resulting signature helps confirm that the token hasn't been tampered with.

These three parts work together to create a JWT, which is a vital tool in ensuring user data stays secure and website interactions remain smooth.

Authentication Flow

Let's look at how JWT actually works in practice step by step:

- Login: You enter your username and password on the website. If the details are correct, the server generates a JWT and sends it back to you.
- *Store and Send:* Your browser then stores this token (often in a place called local storage). Every time you visit a new page on the site or request a protected resource, your browser automatically sends the JWT along in the headers of the HTTP request.
- *Verify and Allow:* The server gets this request, and the JWT with it. It checks the JWT's signature to make sure it's legit and hasn't been messed with. If everything checks out, the server provides the resources you asked for.

In short, JWT helps websites remember who's logged in and keeps your session active as you move around the site.

Implementation

To start with the JWT authentication we will implement an interceptor creating the *authorize.interceptor.ts* file under *Interceptors* folder.

In Angular, Interceptors are like having checkpoints where changes can be made, like adding headers to a request or logging responses from a server.

In our case, we'll use an interceptor to automatically add our JWT to every request that needs it.

For that, we'll need to install a couple of packages y typing the following in our project directory.

```
npm install @auth0/angular-jwt
@angular/common/http
```

Next, we'll create interceptor service class and add a function to take the JWT we stored and add it to the header of any HTTP requests that need it, like this.

```
@Injectable()
2 references
export class HttpInterceptorService implements HttpInterceptor {
  0 references
  constructor(
    private router: Router,
    private userService: UserService,
    private toastersService: ToastersService,
  ) {}

  intercept(
    request: HttpRequest<any>,
    next: HttpHandler
  ): Observable<HttpEvent<any>> {

    const token = localStorage.getItem('Template_jwt');

    if (token) {
      request = this.addToken(request, token);
    }

    return next.handle(request).pipe(
      catchError((error: HttpErrorResponse) => {
        if (error.status === 401) {
          return this.handle401Error(request, next);
        }
        return throwError(error);
      })
    );
  }
```

Tokens often have an expiration time for security reasons. But what happens when a token expires? Usually, the server will return a 401 unauthorised status, as we can see in the source code provided, a strategy is put in place to either refresh the token or prompt the user to log in again.

Finally, we will register this interceptor in the *app.config.ts* file so that the token will be added to all outgoing HTTP requests.

Login

Authentication starts with a Login page, which can be hosted either in our domain or in a third-party domain. In an enterprise scenario, the login page is often hosted on a separate server, which is part of a company-wide Single Sign-On solution.

A separately hosted login page is an improvement security-wise because this way the password is never directly handled by our application code in the first place.

In this article, since the scenario is different due to the solution provided in the source code just hosts the application itself, we will create a component file called *login.component.ts* with the login procedure.

Access Control

Once that we have the authentication procedure implemented, we need to implement the access control with guards and conditional view for UI components.

Route guards are functions that control whether a user can navigate to or leave a particular route. They are like checkpoints that manage whether a user can access specific routes. Common examples of using route guards include authentication and access control.

You can generate a route guard using the Angular CLI:

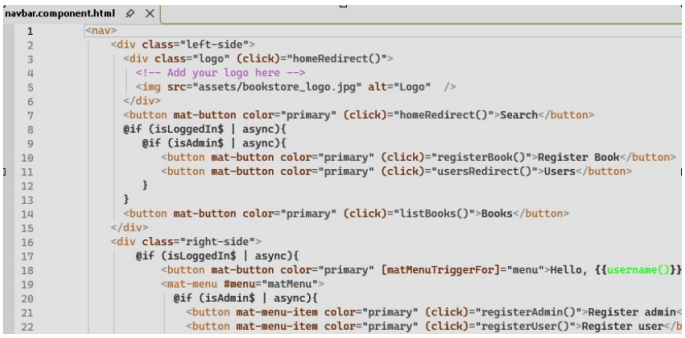
```
ng generate guard auth
```

Then you can place the *auth.guard.ts* file generated under *guards* folder and implement the needed code like this.

Afterwards we will declare the authorisation guard in the *app.routes.ts* like this.

```
{ path: 'user-details/:id', component:
UsersDetailsComponent, canActivate:
[authGuard] }
```

As we created a role based authorisation, together the routes the UI components can be controlled by adding conditional rendering like this.



The code behind this UI component will initialise the variables in the export class like this.

```
isLoggedIn$ =
this.userService.isAuthenticated$;

isAdmin$ = this.userService.isAdmin$;
```